

Why These Six Protocols

The six coordination-protocol arms are not a sample of the protocol space. Each answers a specific, measured failure in how the replication's agents actually talked to each other.

§	Section	Covers	Length
1	The Problem	What 703 replication messages reveal	3 slides
2	The Design	One axis, six rungs — and a prediction	2 slides
3	The Six Arms	Why each was chosen, and what happened	7 slides

1

The Problem

What 703 replication messages reveal

SLIDE 1.1 The agents coordinate — exhaustively, and at the wrong grain

- We extracted **every message** sent in the replication's messaging condition: **703 messages across 147 pair-runs**.
- Each run's dialogue was coded against nine themes, then joined to the **patch-level outcome** of the same run.
- The exchange is a pair of announcements, not a negotiation: a question is asked in **9%** of runs, the median run sends **5 messages**, and the whole conversation is over in **114 seconds** — after which both agents work alone.

TAKEAWAY

They talk. They just don't negotiate — and they stop talking almost immediately.

SLIDE 1.2 Nine themes in the replication's messages

	Theme	Prevalence	Conflict rate
T1	Coordination is file-scoped — every run names the files it will touch	100%	—
T2	Ownership is explicitly proposed (“you take X, I take Y”)	1%	—
T3	Both agents say they will append their parameter last	20%	97%
T4	Sequencing illusion (“I’ll go first”, “layer yours on top”)	24%	77%
T5	Workspace isolation discovered mid-run (“separate sandboxes”)	27%	—
T6	A clean merge is self-certified (“additive”, “no overlap”)	63%	63%
T7	A question is actually asked of the partner	9%	—
T8	Verbatim code is exchanged	3%	—
T9	Ends with a terminal “done / submitting” announcement	50%	—

Prevalence = share of the 147 messaged pair-runs. Conflict rate = share of those runs that nonetheless ended in a textual merge conflict.

TAKEAWAY

Files are named in every run (T1) — but almost nobody allocates them (T2), and almost nobody sends code (T8).

SLIDE 1.3 Where the conflicts actually happen

Patch forensics, conflicting runs	Share
Conflicting runs analysed	93
...both patches touch a shared file	100%
...the colliding file was explicitly named in the chat	100%
...collision starts at the byte-identical same line	90%
...both patches rewrite the same function signature	27%

- Every collision happened inside a file the agents had **already discussed**.
- The information needed to avoid it was **never missing** — it was volunteered in every single run, and it was insufficient.

TAKEAWAY

The conflicts live **below the file**. File-level coordination cannot reach them.

SLIDE 1.4 The dialogue that proves it — pallets_jinja #1465

agent1 “ack — we’re both touching sync_do_groupby... *Let’s agree on final param order/placement to avoid merge clashes.*”

agent2 “*Suggest final combined order: (environment, value, attribute, default=None, separator='.', reverse=False) — just append your reverse param after mine... will let you know when I’m fully done so you can layer your edit on top.*”

agent1 “my working copy doesn’t show your separator changes... we appear to be in **separate sandboxes**, not a shared filesystem. Since each of us submits our own patch.txt independently, *this should be fine — no actual clobbering*. Good luck with yours!”

- agent2 states the **correct merged signature explicitly**. agent1 agrees to it. **Neither writes it**.
- Each emits only its own half into its own checkout — the patches collide at `filters.py:1163`.

TAKEAWAY

The agreement existed as **prose** and never became **bytes**.

2

The Design

One axis, six rungs — and a prediction

SLIDE 2.1 The diagnosis, and the variable it names

The protocol lets agents converge on a description of the merged code, while the merge is decided by the bytes.

- So the six arms sweep exactly one variable: **how much of the final merged artifact the protocol obliges the pair to converge on.**
- Nothing else changes — same model, same tasks, same evaluation, same integration model. Only the protocol block in the prompt and the message-field validation.
- That makes any difference between arms a **protocol effect**, not a model or scaffold effect.

TAKEAWAY

Six rungs on one ladder, not six unrelated ideas.

SLIDE 2.2 The ladder — and the prediction it makes in advance

Rung	Protocol	What the pair must converge on	Merge-clean / both-passed
1	control	Nothing	13% / 2%
2	free-text	Prose intent	21% / 3%
3	semi-structured	Validated metadata about intent	16% / 3%
4	plan-handshake	A binding partition of files	20% / 10%
5	designated-coder	A single writer per shared file	18% / 58%
6	coauthor-overlap	The merged bytes themselves	78% / 69%

- Because the conflicts are measurably **sub-file** (slide 1.3), the ladder is falsifiable *before any arm is run*:
- → arms stopping **at or above file granularity** should not move the merge-clean rate;
- → only arms reaching **the construct** should.

TAKEAWAY

Rungs 1–4 should fail, 5–6 should work. That is exactly what happens.

3

The Six Arms

Why each was chosen, and what happened

SLIDE 3.1 control — no messaging

Must agree: nothing

The floor

- Establishes the conflict rate when the channel is **removed entirely**.
- In 63% of replication runs the pair self-certifies a clean merge (T6). If that talk were load-bearing, deleting it should visibly hurt.
- *Prediction*: if messaging does real work, control should be clearly worse.

RESULT 13% merge-clean · 2% both-passed

TAKEAWAY Free-text beats it by 8 points — **not significant** ($p = 0.105$). The talk was never load-bearing.

SLIDE 3.2 free-text — unconstrained messaging

Must agree: prose intent

The replication's own condition

- Carries the replication protocol down to the screened 18-pair subset.
- Guarantees every other arm is measured against the **same population** in which themes T1–T9 were observed.
- *Prediction*: reproduces the replication's themes and its floor-level merge rate.

RESULT 21% merge-clean · 3% both-passed

TAKEAWAY Reference arm. Confirms the diagnosis transfers to the nano subset.

SLIDE 3.3 semi-structured — typed, validated fields

Must agree: metadata about intent

Targets T1 (100%) · T2 (1%)

- Tests the obvious first hypothesis: the coordination information is **present but informal**.
- Enforces it as typed `--type / --files / --summary` fields; malformed messages are rejected and never delivered.
- *Prediction*: T1 = 100% predicts **failure** — agents already volunteer everything the schema asks for.

RESULT 16% merge-clean · 3% both-passed

TAKEAWAY *Not significant ($p = 1.00$). Rules out “they just needed to say more” — the information was never missing.*

SLIDE 3.4 plan-handshake — agree a disjoint file split

Must agree: a binding partition of files

Targets T2 (1%) · T4 (24%, 77% conflict)

- The pair almost never allocates, and substitutes an **unenforceable turn-taking story**.
- Replaces it with a mutually `ACCEPTED` disjoint split and a **real blocking barrier** (`coop-await`) before any edit.
- *Prediction*: fixes allocation — but the split is still file-level, so merge-clean should not move.

RESULT 20% merge-clean · 10% both-passed

TAKEAWAY *Merge-clean **n.s.**; both-passed 2%→10% ($p = 8 \times 10^{-11}$). The barrier bought **correctness**, not **mergeability** — so the residue is not an allocation failure.*

designated-coder — one owner per shared file

Must agree: a single writer, plus a spec

Targets the file-granularity ceiling

- Median shared files per conflict = 1, and it is always the file **both agents named** — so no disjoint split of it exists.
- Stops splitting: assigns one owner, and routes the other's needs as a written SPEC instead of as conflicting edits.
- *Prediction*: removing the second writer should collapse the conflict rate.

RESULT

18% merge-clean · 58% both-passed

TAKEAWAY

Textual conflict 80%→39%. But the gain arrives as **solo-rescue** (43%) — one patch carrying the pair — and merge-clean stays at the floor.

coauthor-overlap — byte-identical merged code

Must agree: the merged bytes themselves

Targets T3 (20%, 97% conflict) · T8 (3%)

- 90% of conflicts start at the **identical line**; 27% rewrite the same def.
- Pairs **already try** to state the merged construct in prose — they just never exchange it as code (T8 = 3%).
- So it requires that construct to be agreed and then emitted **verbatim, byte-for-byte, in both patches**.
- *Prediction*: the only rung reaching the construct — the only one that should work.

RESULT

78% merge-clean · 69% both-passed

TAKEAWAY

CMH OR 27.7, $p < 10^{-21}$. Produces **26 byte-identical merges** against exactly 1 across the other five arms' 1,168 runs combined.

What the ladder shows

- **Five of six arms fail — in the order the messaging evidence predicts.**
- Enriching the channel (semi-structured) and planning a file-level split (plan-handshake) leave merge-clean at the **no-messaging floor**: neither reaches the granularity at which the patches actually collide.
- Designated-coder concedes the point and removes the second writer, halving textual conflict — but rescues correctness through **one agent carrying the pair**, not through a clean merge.
- Only coauthor-overlap changes **what the merge receives** rather than how much the agents talk. It is the only arm to separate from control on the primary endpoint after Holm correction.

TAKEAWAY

Protocol design works — but only when it changes the bytes, not the conversation.

Message themes and patch forensics: `uv run python masters_thesis/protocol_analysis/replication_messages.py over logs/flash_msg_*`. Arm endpoints and inference: `analyze.py`, frozen in `data/nano_study.json`. This pack: `protocol_rationale_pdf.py`.